Serial Game Server: Protocol Specification v1.1

EEE3095S CS Project 2026 | Tic-Tac-Toe, human vs human | STM32 (C) + PC (Python). Shared reference for Partner A and Partner B. Change only by agreement of both partners.

1. Framing and transport

Item	Decision
Link	UART (USART1: PA9 TX / PA10 RX) via USB-UART adapter or board bridge, 115200 baud, 8N1
Encoding	ASCII, case-sensitive, uppercase commands
Terminator	**\n**. Server ignores **\r**, so \r\n also works
Max frame	32 bytes excluding the terminator
Checksum (optional)	Suffix ***HH**: HH = XOR of all bytes before '*', two hex digits. If present it must be valid, else NACK:CHECKSUM. If absent, frame is accepted. Server replies carry no checksum
Inter-byte timeout	500 ms. Partial frame is discarded and server sends NACK:TIMEOUT
Overflow	More than 32 bytes: send NACK:OVERFLOW once, discard bytes until next \n
Garbage	Non-printable bytes or unknown command at \n: NACK:BADCMD
Response time	Within 100 ms of the terminator

2. Board encoding

Nine characters, index 0-8 row-major (0 = top-left, 8 = bottom-right). '.' empty, 'X' or 'O'. X always moves first.

3. Commands and responses

Request	State	Response
CONNECT	IDLE	OK
CONNECT	other	NACK:BUSY
NEW	IDLE / GAME_OVER	OK:BOARD:..... (enter IN_PROGRESS, human vs human)
NEW:AI	IDLE / GAME_OVER	OK:BOARD:..... (human is X, server plays O)
NEW / NEW:AI	IN_PROGRESS	NACK:BUSY
MOVE:<i> or MOVE:<M>:<i>	IN_PROGRESS	OK:BOARD:<9> | WIN:<M>:<9> | DRAW:<9> | NACK:*
MOVE:...	IDLE / GAME_OVER	NACK:NOGAME
STATE	IN_PROGRESS / GAME_OVER	BOARD:<9>:<turn>:<status>
STATE	IDLE	NACK:NOGAME
HIST	IN_PROGRESS / GAME_OVER	HIST:<n>:<moves> e.g. HIST:3:X4O0X8
HIST	IDLE	NACK:NOGAME
RESIGN	IN_PROGRESS	OK:GAMEOVER (enter GAME_OVER)
RESIGN	other	NACK:NOGAME
DISCONNECT	any	OK (enter IDLE)
anything else	any	NACK:BADCMD

<M> = X or O. <turn> = X, O or '-'. <status> = IDLE, PLAY or OVER. Unsolicited: **EVT:RESET** (button press).

Computer opponent (NEW:AI). The human is always X and moves first; MOVE:O:<i> returns NACK:NOTURN. After each legal human move that does not end the game, the server picks a random empty cell, plays it through the same validation, and appends **:AI:<cell>** to the reply: **OK:BOARD:<9>:AI:<c>**, or **WIN:O:<9>:AI:<c>** / **DRAW:<9>:AI:<c>** if the computer's move ends the game. If the human's move ends the game (WIN:X or DRAW) there is no suffix. The board in the reply already includes the computer's move. Plain NEW games are unchanged.

History (HIST). <n> is the number of moves (0-9); <moves> is two characters per move, mark then cell, in play order, including computer moves (HIST:0: when empty). Rejected moves are never recorded. History is kept after GAME_OVER and cleared by NEW, DISCONNECT and the button reset. Suggested GUI use: call HIST after every accepted reply and display the list.

4. MOVE validation order (both sides implement identically)

1. Frame and checksum valid? Else NACK:CHECKSUM / NACK:OVERFLOW / NACK:TIMEOUT.
2. Known command and argument syntax? Else NACK:BADCMD (e.g. MOVE:abc).
3. State is IN_PROGRESS? Else NACK:NOGAME.
4. Mark matches current turn (if given)? Else NACK:NOTURN.
5. Index in 0-8? Else NACK:RANGE (MOVE:9, MOVE:-1).
6. Cell empty? Else NACK:OCCUPIED.
7. Only now mutate the board, then check win / draw.

Steps 1-6 never write game state. This is the 'rejected requests cannot change state' argument for the report.

5. Button and LED

LED: steady on = X's turn, off = O's turn, 2 Hz blink = GAME_OVER. **Button:** debounced press forces IDLE and sends **EVT:RESET**. Any line starting with **EVT:** is asynchronous: the Python serial wrapper must queue or skip it, never treat it as a reply.

6. Departures from the reference brief (for report section 2)

• **WIN:X:<board> / DRAW:<board>** instead of bare WIN:X / DRAW, so the GUI can redraw the final board from a server response.
• **Optional mark in MOVE:<M>:<i>** makes NACK:NOTURN testable and keeps the client untrusted. Bare MOVE:<i> still works.
• **STATE carries turn and status** so the GUI can resync after reconnect or button reset.
• **New errors:** NACK:BUSY, TIMEOUT, OVERFLOW, CHECKSUM, for specificity beyond BADCMD.
• **DISCONNECT valid in any state** so a crashed GUI cannot leave the server stuck.

7. Example session

```
>CONNECT        <OK
>NEW            <OK:BOARD:.........
>MOVE:X:4       <OK:BOARD:....X....
>MOVE:X:0       <NACK:NOTURN
>MOVE:O:4       <NACK:OCCUPIED
>MOVE:O:9       <NACK:RANGE
>MOVE:O:0       <OK:BOARD:O...X....
 ... X completes 2,4,6 ... <WIN:X:O.X.XOX..  (example)
>MOVE:O:1       <NACK:NOGAME
>DISCONNECT     <OK
```

Example: computer opponent

```
>NEW:AI         <OK:BOARD:.........
>MOVE:X:4       <OK:BOARD:..O.X....:AI:2
>HIST           <HIST:2:X4O2
```